

Flyweight

Use sharing to support large numbers of fine-grained objects efficiently — split state into shared intrinsic and per-use extrinsic.

📍

DIVE 01 · 10 km

Airplane porthole

The big picture & contours

→

🏠

DIVE 02 · 1 km

Train window

Theses, examples, terms

→

♀

DIVE 03 · 100 m

Park walk

How the pieces relate

→

🔍

DIVE 04 · 1 cm

Microscope

Exact wording & gotchas

📍

Airplane porthole

DIVE 01 · ORIENTATION

▼ 10 km

A memory pattern — share one immutable copy of the state objects have in common, and pass the rest in from outside.

THE CONTOURS

• Intrinsic vs extrinsic

shared, context-free state vs per-instance state passed in

• Cache & reuse

a factory hands back the one shared flyweight per key

• Trades CPU for RAM

far fewer objects, but context must be supplied each call

USE IT WHEN — GoF applicability

🎯 A huge number of objects

would blow memory if each were a full, standalone instance

📦 State splits cleanly

into shared intrinsic data and contextual extrinsic data

📦 Structural family: Flyweight shares leaves · Composite arranges them in a tree · Proxy stands in for one · Decorator wraps one. Flyweights are often a Composite's leaves.

↓ DIVE DEEPER

🏠

Train window

DIVE 02 · KEY OBJECTS

▼ 1 km

THE THREE PARTS

Flyweight

stores intrinsic state; its operations take extrinsic state as args

FlyweightFactory

creates & pools flyweights, returning shared ones by key

Client

holds the extrinsic state and passes it into flyweight operations

IN THE WILD

✳️ Char glyphs in editors

one object per character code, shared across the doc — GoF's example

🎮 Game sprites / particles

one mesh shared by thousands of trees, position passed in

💻 Java Integer cache

Integer.valueOf(-128..127) returns shared boxed instances

🗄️ String interning

JS / Java pool identical string literals to one object

<> Client → Factory.get(key) → shared Flyweight; the client calls flyweight.op(extrinsic).

GoF intent: “Use sharing to support large numbers of fine-grained objects efficiently.”

↓ DIVE DEEPER

♀

Park walk

DIVE 03 · CONNECTIONS

▼ 100 m

Client

needs object

→

Factory.get(key)

pool lookup

→

shared flyweight

intrinsic state

→

✓ op(extrinsic)

context passed in

WHY IT FOLLOWS

→ Split the state ⇒ only the shared part lives in the object; the rest is supplied per call

→ Pool by key ⇒ identical requests return the same instance, never a new one

→ Immutable shared part ⇒ safe to share across every context without copies

🔗 Flyweights are usually a Composite's leaf nodes; the factory is often a Singleton. State & Strategy objects can be shared as flyweights. Contrast an object pool — that recycles, this shares.

STRUCTURE · UML

Flyweight

- intrinsic: State

+ operation(extrinsic): void

Operations receive extrinsic state as parameters; the factory guarantees one shared instance per intrinsic key.

WHAT IT BUYS YOU

🟢 Massive memory savings — thousands of references to a handful of shared objects

🟢 More objects feasible — represent things you couldn't afford as full instances

🟢 Centralized state — intrinsic data is defined once, in one place

🟢 Run-time cost — extrinsic state must be computed or stored and passed each call

🟢 Needs immutability — shared intrinsic state must never be mutated by a client

↓ DIVE DEEPER

🔍

Microscope

DIVE 04 · DETAILS

▼ 1 cm

Definition: “Use sharing to support large numbers of fine-grained objects efficiently.”
— GoF, “Design Patterns”, 1994

THE CODE

```
class Glyph { // intrinsic state, shared
  constructor(readonly char) {}
  draw(x, y, font) { // extrinsic args
    paint(this.char, x, y, font);
  }
}
const pool = new Map();
function glyph(c) { // the factory
  if (!pool.has(c)) pool.set(c, new Glyph(c));
  return pool.get(c);
}
```

VARIANTS

Shared flyweight

pooled & reused across contexts — the core case

Unshared flyweight

a leaf that happens not to be shared (GoF allows it)

Intrinsic / extrinsic

the defining split — context-free vs contextual state

Interning

string / symbol pools — equal values share one object

Value-object cache

Integer.valueOf, boxed primitives in a range

Object pool (contrast)

reuses mutable instances — recycling, not sharing

COMPARE THE ALTERNATIVES

Property	Flyweight	Singleton	Obj pool	Cache
Many shared instances	✓	✗	✓	✓
Splits intrinsic / extrinsic	✓	✗	✗	✗
Shared state immutable	✓	~	✗	~
Goal is saving memory	✓	~	✓	~
Same object by key	✓	✗	✗	✓

Flyweight shares immutable atoms · Pool recycles mutable ones · Cache stores results · Singleton allows exactly one.

INTERVIEW PITFALLS

Q “Intrinsic vs extrinsic state?”

Intrinsic is shared & context-free (lives in the flyweight); extrinsic is per-use, passed in by the client.

Q “Flyweight vs object pool?”

Flyweight shares immutable objects concurrently; a pool lends out and recycles mutable ones one at a time.

Q “Why must shared state be immutable?”

Because one object backs many contexts — any mutation leaks into all of them.

Q “Is String interning a flyweight?”

Yes — equal literals share one immutable instance handed out by a pool.

NUANCES & GOTCHAS

Shared state must be immutable — a client mutating intrinsic data corrupts every context

Extrinsic state costs — you trade RAM for the CPU / storage of supplying context

Identity loses meaning — == no longer implies “same logical thing”; many share one object

Factory is mandatory — clients must not new flyweights directly, or sharing breaks

Only worth it at scale — for a few objects the bookkeeping costs more than it saves

Watch the pool size — an unbounded pool is a leak; bound it or weakly reference

WATCH OUT

⚠️ Anti-pattern: Skip it unless object count is genuinely huge and state splits cleanly into shared vs contextual — otherwise the factory, the extrinsic plumbing and the lost identity cost more than the RAM saved.

⚡ Modern take: rarely hand-rolled today — engines do it for you (string interning, boxed-int caches, GPU instancing). Reach for it in hot paths with millions of look-alike objects.

• Vasyl Krupka · Senior Fullstack Engineer · learn-by-diving series

Source: GoF “Design Patterns” (1994) v1